End-to-End Medical Chatbot (RAG)

Interview Preparation Guide - Questions & Detailed Answers

This document covers project architecture, Retrieval-Augmented Generation (RAG) concepts, LangChain, embeddings, Pinecone vector database, Flask, prompt engineering, Docker, and AWS CI/CD deployment questions relevant to this project, along with detailed explanations.

1. Project Overview & Architecture

Q: What is this project and what problem does it solve?

A: This is an End-to-End Medical Chatbot built using Retrieval-Augmented Generation (RAG). It answers medical/health-related questions by retrieving relevant context from a knowledge base (PDF documents such as medical encyclopedias) stored as vector embeddings in Pinecone, and then passes that retrieved context to an OpenAI GPT LLM to generate a grounded, concise answer. This avoids the LLM hallucinating answers and keeps responses factually tied to the source documents.

Q: Can you explain the end-to-end architecture/data flow of this application?

A: The flow has two phases: (1) Indexing phase (store_index.py) - PDF files are loaded from the Data/ folder using DirectoryLoader + PyPDFLoader, split into 500-character chunks (20 char overlap) using RecursiveCharacterTextSplitter, converted into 384-dimension embeddings via a HuggingFace sentence-transformers model, and upserted into a Pinecone serverless index named 'medicalbot'. (2) Query phase (app.py) - Flask serves a chat UI; a user message hits the /get route, the same embedding model embeds the query, Pinecone's retriever returns the top-k (k=3) similar chunks, LangChain's create_stuff_documents_chain 'stuffs' those chunks into a prompt template along with the system prompt, and create_retrieval_chain sends it to the OpenAI LLM which generates the final answer returned to the browser.

Q: Why use RAG instead of just fine-tuning or prompting the LLM directly?

A: RAG lets you ground the model's answers in your own up-to-date, domain-specific documents without the cost, time, and retraining overhead of fine-tuning. It reduces hallucination because the model is instructed to answer using retrieved context, it's easy to update knowledge (just re-index new PDFs), and it's cheaper/faster to iterate than fine-tuning a large model.

Q: What are the main components/technologies used in this stack and why?

A: Python (core language), LangChain (orchestration framework connecting loaders, splitters, embeddings, vector store, retriever and LLM into chains), Flask (lightweight web server/API and UI), OpenAI GPT (the generative LLM), Pinecone (managed serverless vector database for similarity search), HuggingFace sentence-transformers (local, free embedding model), python-dotenv (secret/env management), Docker (containerization for consistent deployment), and GitHub Actions + AWS EC2/ECR (CI/CD deployment).

Q: Walk me through the folder structure of the project.

A: src/helper.py holds reusable functions (PDF loading, text splitting, embeddings). src/prompt.py holds the system prompt. store_index.py is a one-time/offline script to build the Pinecone index from the Data/ folder. app.py is the Flask web app that serves the chat UI and answers queries at runtime. templates/chat.html and static/style.css hold the front-end. template.py is a bootstrap script used at project creation time to scaffold the folder/file structure. setup.py packages the project (src as an installable package) and requirements.txt lists dependencies, including '-e .' to install the local package in editable mode.

2. LangChain Concepts

Q: What is LangChain and what role does it play here?

A: LangChain is a framework for building applications powered by LLMs. It provides standard abstractions for document loaders, text splitters, embeddings, vector stores, retrievers, prompt templates, and 'chains' that compose these pieces together. In this project it wires together the Pinecone retriever, the ChatPromptTemplate, and the OpenAI LLM into a single retrieval-augmented question-answering chain.

Q: What is the difference between create_stuff_documents_chain and create_retrieval_chain?

A: create_stuff_documents_chain builds a chain that takes a list of retrieved Documents and 'stuffs' (i.e., concatenates) their text content directly into the {context} placeholder of the prompt before sending it to the LLM - suitable when the combined context fits within the model's context window. create_retrieval_chain wraps a retriever and a document-combining chain (like the stuff chain) together: given a user {input}, it first calls the retriever to fetch relevant documents, then passes those documents plus the input into the document chain to produce the final answer, returning both the answer and the source documents.

Q: What is a retriever in LangChain, and what does search_type='similarity' and k=3 mean?

A: A retriever is an interface that returns relevant Documents for a query, abstracting away the underlying vector store. Here docsearch.as_retriever(search_type='similarity', search_kwargs={'k':3}) means it does a standard vector similarity search (cosine similarity via Pinecone) and returns the top 3 most similar chunks to the user's query embedding.

Q: What is a PromptTemplate/ChatPromptTemplate and why is it used?

A: ChatPromptTemplate.from_messages() defines a structured, reusable prompt with role-based messages (a 'system' message containing instructions/context placeholder, and a 'human' message containing the user's {input}). It ensures consistent formatting when injecting dynamic values (like retrieved context and the user's question) into the prompt sent to the LLM, and keeps prompt engineering separate from application logic.

Q: What text splitter is used and why is chunking necessary?

A: RecursiveCharacterTextSplitter with chunk_size=500 and chunk_overlap=20. Chunking is necessary because embedding models and LLMs have limited context windows, and because retrieval works best when chunks are small and semantically focused (a whole document is too coarse for accurate similarity search). RecursiveCharacterTextSplitter tries to split on natural boundaries (paragraphs, sentences, words) before falling back to hard character limits, preserving semantic coherence. The 20-character overlap ensures context isn't abruptly cut off between adjacent chunks, so information spanning a chunk boundary isn't lost.

Q: What document loaders are used to ingest the data?

A: DirectoryLoader scans the Data/ directory for files matching a glob pattern (*.pdf), and PyPDFLoader (passed as loader_cls) extracts text from each PDF page-by-page into LangChain Document objects, each carrying page_content and metadata (like source file and page number).

Q: How would you extend this chain to also return the retrieved source documents to the user?

A: create_retrieval_chain's response dict already includes both 'answer' and 'context' (the retrieved Documents) keys. You could modify the /get Flask route to also return response['context'] (e.g., the source file/page metadata) alongside the answer, letting users see which document snippets were

used - improving transparency and trust, which is especially important in a medical context.

3. Embeddings & Vector Database

Q: What embedding model is used and why?

A: sentence-transformers/all-MiniLM-L6-v2 from HuggingFace, loaded via HuggingFaceEmbeddings. It produces 384-dimension dense vectors. It's chosen because it's small, fast, free to run locally/offline (no API cost per call unlike OpenAI embeddings), and provides good general-purpose semantic similarity performance for retrieval tasks.

Q: What is an embedding, conceptually?

A: An embedding is a numeric vector representation of text (or other data) in a high-dimensional space, positioned so that semantically similar pieces of text have vectors that are close together (measured by metrics like cosine similarity or Euclidean distance). This enables 'similarity search' - finding text with similar meaning rather than just exact keyword matches.

Q: Why use Pinecone instead of storing embeddings in a plain database or in-memory structure?

A: Pinecone is a managed, serverless vector database purpose-built for high-dimensional similarity search at scale. It provides fast approximate nearest neighbor (ANN) search, horizontal scalability, persistence, metadata filtering, and a simple API - without needing to manage infrastructure. A plain SQL/NoSQL database isn't optimized for high-dimensional nearest-neighbor search and would be far slower or require significant custom indexing work at scale.

Q: What is 'cosine similarity' and why is it used as the metric here?

A: Cosine similarity measures the cosine of the angle between two vectors, focusing on their direction rather than magnitude - a value near 1 means the vectors point in nearly the same direction (highly similar meaning). It's commonly used for text embeddings because embedding magnitude can vary with text length, but direction better captures semantic similarity.

Q: What does dimension=384 mean when creating the Pinecone index, and why must it match the embedding model?

A: It declares the length of each vector Pinecone will store and index. It must exactly match the output dimensionality of the embedding model (all-MiniLM-L6-v2 outputs 384-dim vectors) - Pinecone cannot store or compare vectors of mismatched dimensions, so index creation and the embedding model must be kept in sync.

Q: What is ServerlessSpec and what does specifying cloud='aws', region='us-east-1' mean?

A: ServerlessSpec configures a Pinecone serverless index, meaning Pinecone auto-manages the underlying compute/storage and scales on demand (pay-per-use) rather than requiring you to provision fixed pods. The cloud and region parameters specify where (which cloud provider and geographic region) the index's underlying infrastructure is hosted, which affects latency and data residency.

Q: What's the difference between Pinecone.from_documents() and Pinecone.from_existing_index()?

A: from_documents() (used in store_index.py) embeds a list of Document chunks and upserts them into a (newly created) index - used for the one-time ingestion/indexing step. from_existing_index() (used in app.py) simply connects to an index that already has vectors stored in it, without re-embedding or re-uploading anything - used at query/runtime since the index was already populated.

Q: How would you handle updating the knowledge base when new medical PDFs are added?

A: Re-run (or schedule) the ingestion pipeline: load only the new/changed PDFs, split them into chunks, embed them, and upsert into the same Pinecone index (Pinecone upserts are idempotent by vector ID). For production, you'd want to track already-ingested files/hashes to avoid duplicate embeddings, and possibly delete/replace vectors tied to updated documents.

4. Prompt Engineering & LLM

Q: What is the system prompt doing in this project?

A: It instructs the LLM to act as a question-answering assistant that must use only the retrieved context to answer, admit when it doesn't know an answer, and keep responses concise (max three sentences). This constrains the model's behavior to reduce hallucination and keep answers focused and safe, which is especially important for a medical-domain assistant.

Q: Why set temperature=0.4 for the OpenAI LLM here?

A: Temperature controls randomness/creativity in generation - lower values produce more deterministic, focused, and factual outputs, while higher values produce more varied/creative ones. A relatively low value like 0.4 is chosen for a medical Q&A; assistant to favor consistent, factual, less speculative answers over creative variation.

Q: What does max_tokens=500 control, and what are the trade-offs of setting it?

A: It caps the maximum length of the LLM's generated response (in tokens). Setting it too low risks cutting off answers mid-sentence; setting it too high increases latency and cost per call. 500 is a reasonable bound for concise, few-sentence medical answers as instructed by the system prompt.

Q: How does the {context} placeholder get filled in the prompt?

A: create_stuff_documents_chain automatically formats the retrieved Document objects (from the retriever) and inserts their concatenated page_content into the {context} variable inside the system message before sending the full prompt to the LLM.

Q: What risks exist with this prompting approach, and how would you mitigate them?

A: Risks include: hallucination if retrieval returns irrelevant chunks, exposing incorrect medical advice, prompt injection via malicious user input, and leaking sensitive data if PDFs contain PII. Mitigations: stronger system instructions with explicit refusal/disclaimer patterns ('not a substitute for professional medical advice'), input sanitization, output filtering/moderation, restricting retrieval to trusted sources only, and logging/monitoring responses for review.

Q: How would you evaluate the quality of this RAG chatbot's answers?

A: Techniques include: building a labeled QA test set from the source PDFs and measuring answer accuracy/relevance (manually or with an LLM-as-judge), evaluating retrieval quality separately (e.g., recall@k - does the correct chunk appear in top-k results), tracking faithfulness/groundedness (does the answer only use retrieved content), and using frameworks like RAGAS or LangSmith for automated RAG evaluation metrics (context precision, context recall, answer relevancy, faithfulness).

5. Flask & Application Layer

Q: Explain the Flask routes defined in app.py.

A: The '/' route (GET) renders chat.html, the front-end chat UI. The '/get' route (accepts GET and POST) reads the user's message from request.form['msg'], invokes the RAG chain with rag_chain.invoke({'input': msg}), and returns the plain-text answer (response['answer']) back to the front-end, which is typically called asynchronously via JavaScript/AJAX from chat.html.

Q: Why is the RAG chain (embeddings, retriever, LLM, prompt) built once at module load time instead of per-request?

A: Building embeddings objects, connecting to the Pinecone index, and constructing the chain involves relatively expensive setup (model loading, network/client initialization). Doing this once when the Flask app starts (at import time) and reusing the same objects for every request avoids repeating that overhead on every API call, significantly improving response latency.

Q: What security/production concerns exist with the current Flask setup (debug=True, host='0.0.0.0')?

A: debug=True should never be used in production - it enables the interactive Werkzeug debugger which can allow arbitrary remote code execution if exposed, and it leaks stack traces/source code on errors. host='0.0.0.0' binds to all network interfaces, which is fine behind a properly firewalled/reverse-proxied deployment but risky if directly exposed to the internet without additional protection. For production, you'd disable debug, run behind a WSGI server (gunicorn/uWSGI) with a reverse proxy (nginx), add HTTPS, rate limiting, and authentication if needed.

Q: How are secrets like API keys managed in this project?

A: Via python-dotenv's load_dotenv(), which reads a local .env file (not committed to source control) into environment variables, then os.environ.get() retrieves PINECONE_API_KEY and OPENAI_API_KEY at runtime. In the CI/CD deployment, these are instead injected as GitHub Actions repository secrets and passed to the runtime environment (e.g., EC2/Docker container) rather than being hardcoded.

Q: How would you add conversational memory (multi-turn context) to this chatbot?

A: Introduce a memory component (e.g., LangChain's conversation history / message history classes) that stores prior turns per session (e.g., keyed by a session ID or Flask session cookie), and include recent conversation history in the prompt alongside the retrieved context, or use a history-aware retriever that reformulates the user's question using prior turns before performing retrieval, ensuring follow-up questions resolve correctly (e.g., using create_history_aware_retriever).

6. Deployment, Docker & CI/CD

Q: What does the Dockerfile do?

A: It starts from a slim Python 3.10 base image, sets /app as the working directory, copies the entire project into the container, installs dependencies from requirements.txt via pip, and defines the container's entrypoint command as 'python3 app.py' so the Flask app starts when the container runs.

Q: Why use Docker for this project?

A: Docker packages the application with all its dependencies and runtime environment into a single portable image, ensuring it runs identically across development, testing, and production (avoiding 'works on my machine' issues). It also simplifies deployment to cloud services/container registries and makes scaling and orchestration (e.g., with ECS/Kubernetes) straightforward.

Q: Describe the AWS CI/CD deployment pipeline used for this project.

A: An IAM user is created with EC2 and ECR access. A Docker image of the app is built and pushed to an AWS ECR (Elastic Container Registry) repository via GitHub Actions. An EC2 (Ubuntu) instance is provisioned with Docker installed and configured as a self-hosted GitHub Actions runner. On each push/deployment trigger, GitHub Actions builds the image, pushes it to ECR, then the self-hosted runner on EC2 pulls the latest image from ECR and runs/launches it as a container, effectively automating build-push-deploy.

Q: Why use a self-hosted GitHub Actions runner on EC2 instead of GitHub-hosted runners?

A: A self-hosted runner executes directly on the target EC2 instance, so it can pull the Docker image from ECR and restart the container locally without needing SSH access management from GitHub's infrastructure, simplifies networking/permissions (the runner already has AWS credentials/IAM role access on the instance), and can be more cost-effective/faster for repeated deployments to a fixed environment.

Q: What IAM permissions are required for this deployment and why?

A: AmazonEC2ContainerRegistryFullAccess (to push/pull Docker images to/from ECR) and AmazonEC2FullAccess (to manage/launch the EC2 instance that hosts the app). These follow the principle of least privilege scoped to only what's needed for the CI/CD pipeline (though in production you'd typically scope these down further to only the specific resources needed rather than full access).

Q: What secrets need to be configured in GitHub Actions for this pipeline, and why keep them as secrets?

A: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_DEFAULT_REGION, ECR_REPO, PINECONE_API_KEY, and OPENAI_API_KEY. They're stored as encrypted GitHub Actions secrets (rather than in code or workflow YAML) so sensitive credentials are never exposed in source control, logs, or pull requests, reducing the risk of credential leakage and unauthorized API/cloud usage/billing.

Q: How would you improve the scalability or reliability of this deployment beyond a single EC2 instance?

A: Options include: putting the container behind an Application Load Balancer with an Auto Scaling Group or moving to a managed container orchestrator (ECS Fargate or EKS) for automatic scaling and self-healing, adding health checks, using a managed secrets store (AWS Secrets Manager/Parameter Store) instead of plain env vars, adding centralized logging/monitoring (CloudWatch), and separating the embedding/indexing pipeline from the always-on query-serving

app so they can scale independently.

7. General / Behavioral & Design Questions

Q: What would you do differently if you were redesigning this project today?

A: Potential improvements: swap OpenAI's completion-style LLM for a chat-optimized model, add streaming responses for better UX, add conversation memory, add source-citation display, add input/output content moderation given the medical domain, add automated RAG evaluation (RAGAS/LangSmith) in CI, containerize the ingestion pipeline separately, and add a proper front-end framework instead of a single static HTML page, plus add unit/integration tests (the placeholder test.py currently isn't implemented).

Q: What are the limitations of this chatbot, and what disclaimers/safeguards should exist given it's medical?

A: It can only answer based on the specific PDFs it was indexed on - it's not a certified medical diagnostic tool, may retrieve irrelevant chunks for out-of-scope questions, could still hallucinate if context is weak, and doesn't understand patient-specific context (allergies, history). It should always include a clear disclaimer that it does not replace professional medical advice, and ideally include escalation guidance to consult a real healthcare provider for serious symptoms.

Q: How does chunk_overlap help versus using chunk_size alone with no overlap?

A: Without overlap, information that spans the boundary between two chunks (e.g., a sentence split across chunk 1's end and chunk 2's start) could lose coherence, since each chunk is embedded and retrieved independently. A small overlap (20 characters here) preserves some shared context between adjacent chunks, improving the chance that key information isn't fragmented/lost during retrieval.

Q: If retrieval returns irrelevant chunks for a query, how would you debug and fix it?

A: First inspect the retrieved documents (response['context']) to see what was actually retrieved. Possible fixes: tune chunk_size/overlap for better semantic granularity, increase/decrease k, try a different/stronger embedding model, add metadata filtering to restrict search scope, use hybrid search (keyword + vector) for exact-term queries, or add a re-ranking step (e.g., a cross-encoder) on top of the initial vector search results to improve precision.

Q: Why keep src/ as an installable package (setup.py with '-e .') rather than plain scripts?

A: Packaging src/ as an installable, editable package makes its modules (helper functions, prompts) importable cleanly from anywhere in the project (e.g., 'from src.helper import ...') without relative-import or PYTHONPATH issues, keeps the codebase organized/modular, and mirrors production-grade Python project structure that's easier to test, package, and distribute.